

# Equation Guide for the Sentinel Report and Slides

Md. Shifat Hasan, Roll 2107067 | CSE 4102

Each section explains one group of equations from Chapter III and the slides: what it does, what each symbol means, why it has that shape, a numeric example, where it lives in the code, and a short spoken summary. All formulas were checked against the source files. Part A (T1–T6) covers *transformations*, how objects are created and combined; Part B (G1–G4) noise, textures and shapes; Part C (L1–L5) *illumination models and lights*; Part D (G5–G10) fog, tone mapping, sky and world.

## Part A. Transformations

### T1. Homogeneous coordinates and the three basic matrices

$$T(\mathbf{t}) = \begin{pmatrix} I & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}, \quad S(\mathbf{s}) = \text{diag}(s_x, s_y, s_z, 1),$$
$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

**Meaning.** A point  $(x, y, z)$  is stored as  $(x, y, z, 1)^\top$ . With that extra 1, *moving* (adding a vector) becomes a matrix multiplication like turning and scaling, so any chain of moves, turns and stretches collapses into a single  $4 \times 4$  matrix. **Symbols:**  $\mathbf{t}$  the move vector;  $\mathbf{s}$  the per-axis stretch;  $\theta$  the turn angle (radians).  $R_x$  and  $R_z$  follow the same pattern about their axes.

#### Worked example (numbers checked by script)

$R_y(90^\circ)$  applied to  $(1, 0, 0)$ :  $x' = x \cos \theta + z \sin \theta = 0$ ,  $z' = -x \sin \theta + z \cos \theta = -1$ , so the point goes to  $(0, 0, -1)$ .

**Code:** Engine/Procedural/Assembly.odin, part\_transform.

#### How to say it in 20 seconds

“A fourth coordinate of one lets a move be a matrix too, so any sequence of moves, turns and scales is one matrix.”

### T2. The transform of one part, and why the order matters

$$M_{\text{part}} = T(\mathbf{t}) R_z(\gamma) R_y(\beta) R_x(\alpha) S(\mathbf{s}), \quad \mathbf{p}_{\text{object}} = M_{\text{part}} \mathbf{p}_{\text{part}}$$

**Meaning.** Matrices multiply the point from the right, so the *rightmost* acts first: the part is stretched, then turned about  $x$ , then  $y$ , then  $z$ , and finally moved. Because the move comes last, the turn happens about the part’s own centre. Rotations and scaling do not commute, so the order is part of the definition.

#### Worked example (numbers checked by script)

Vertex  $(1, 0, 0)$ ,  $\mathbf{s} = (2, 1, 1)$ ,  $\beta = 90^\circ$ ,  $\mathbf{t} = (5, 0, 0)$ : scale gives  $(2, 0, 0)$ ;  $R_y(90^\circ)$  gives  $(0, 0, -2)$ ; the move gives  $(5, 0, -2)$ .

**Code:** Engine/Procedural/Assembly.odin. **Say:** “Scale first, turn about the centre, move last, because the right-most matrix acts first.”

### T3. Normals use the inverse transpose

$$\hat{n}' = \frac{(L^{-1})^\top \hat{n}}{\|(L^{-1})^\top \hat{n}\|}, \quad L = \text{upper-left } 3 \times 3 \text{ block of } M$$

**Meaning.** A normal must stay perpendicular to its surface. Stretching a surface changes its tangent directions, so multiplying the normal by  $M$  would tilt it. The inverse transpose is exactly the matrix that keeps  $\hat{n}' \cdot (L\mathbf{t}) = \hat{n} \cdot \mathbf{t} = 0$  for every tangent  $\mathbf{t}$  (proof:  $((L^{-1})^\top \hat{n})^\top L\mathbf{t} = \hat{n}^\top L^{-1}L\mathbf{t} = \hat{n}^\top \mathbf{t}$ ). The result is re-normalised to unit length. If  $\det L < 0$  the object is mirrored, so the engine also reverses each triangle's winding and the tangent handedness.

#### Worked example (numbers checked by script)

$S = \text{diag}(2, 1, 1)$ ,  $\hat{n} = (1, 1, 0)/\sqrt{2}$ , tangent  $(1, -1, 0) \rightarrow (2, -1, 0)$ . Correct:  $(0.5, 1, 0)/1.118 = (0.447, 0.894, 0)$ , dot with the stretched tangent = 0. Naive  $S\hat{n} \propto (0.894, 0.447, 0)$  gives  $1.34 \neq 0$  (tilted).

**Code:** Engine/Procedural/MeshBuilder.odin, Mesh\_Append. **Say:** “Normals transform by the inverse transpose so they stay perpendicular after a stretch.”

### T4. Hierarchies: products down the chain

$$M_{\text{gun}} = \underbrace{T(\mathbf{p}) R_y(\psi) R_x(-\phi) R_z(\rho)}_{\text{hull}} \underbrace{T(\mathbf{c}_t) R_y(\theta_t)}_{\text{turret}} \underbrace{T(\mathbf{c}_g) R_x(-\theta_g)}_{\text{gun}}$$

**Meaning.** A child part's world matrix is its parent's matrix times its own local matrix.  $\mathbf{p}, \psi, \phi, \rho$ : hull position, yaw, pitch, roll;  $\mathbf{c}_t, \theta_t$ : turret pivot and turn;  $\mathbf{c}_g, \theta_g$ : gun pivot and elevation. Turning the hull changes the leftmost factor, so the turret and gun are carried along, while turning the turret alone leaves the hull factor unchanged. Soldiers work the same way: each limb matrix puts its origin at the upper joint and its local y axis along (lower joint – upper joint). **Code:** Game/Vehicles/Vehicle.odin (Vehicle\_Hull/Turret/Gun\_Matrix); Game/Characters/Soldier.odin (Segment\_Matrix). **Say:** “Each child multiplies its parent's matrix, so moving the parent moves everything below it.”

### T5. Combining parts, placing and instancing

$$M_{\text{world}} = T(x, \text{ground}, z) R_y(\text{yaw}), \quad \mathbf{p}_{\text{world}} = M_{\text{world}} M_{\text{part}} \mathbf{p}$$

**Meaning.** Every part of an object that shares a material is appended into *one* mesh, each vertex multiplied by its  $M_{\text{part}}$  (so one draw call per material). The finished object is placed by  $M_{\text{world}}$ ; *instancing* draws that mesh many times, each with its own  $M_{\text{world}}$  in an instance buffer (one call for thousands of trees). **Code:** Engine/Procedural/Assembly.odin (Assembly\_Build); Game/Base/BaseScene.odin. **Say:** “Bricks are merged by material, then the whole object is stamped many times by one matrix per copy.”

### T6. From the world to the screen: view, projection, zoom

$$\mathbf{p}_{\text{clip}} = P V M_{\text{world}} M_{\text{part}} \mathbf{p}, \quad P = \begin{pmatrix} f/a & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{n+r}{n-r} & \frac{2nr}{n-r} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \quad f = \frac{1}{\tan(\text{fov}/2)}, \quad \mathbf{p}_{\text{screen}} = \frac{(x, y)}{w}$$

$$\tan \frac{\text{fov}'}{2} = \frac{1}{k} \tan \frac{\text{fov}}{2}$$

**Meaning.**  $V$  is the look-at matrix: it moves the camera to the origin and turns the target onto  $-z$ .  $P$  is the

perspective matrix:  $a$  aspect ratio,  $n, r$  near and far planes,  $f$  the focal factor. The fourth output  $w$  equals the distance in front of the camera, and dividing  $(x, y)$  by  $w$  makes far things smaller. A zoom of factor  $k$  narrows the field of view as shown.

**Worked example (numbers checked by script)**

fov = 72°:  $f = 1/\tan 36^\circ = 1.376$ . Binocular zoom  $k = 4$ :  $\tan(\text{fov}'/2) = 0.7265/4$ , so  $\text{fov}' = 20.6^\circ$ .

**Code:** Engine/Render/Camera.odin. **Say:** “Model, view, projection, then divide by depth: that is the whole path from a vertex to a pixel.”

## Part B. Noise, textures and shapes

### G1. Noise deformer

$$\mathbf{p}' = \mathbf{p} + A \nu(f \mathbf{p}) \hat{n}, \quad \|\mathbf{p}' - \mathbf{p}\| \leq A, \quad \nu \in [-1, 1]$$

Every vertex is pushed along its normal by a smooth random amount.  $A$ : amplitude (m);  $f$ : cycles per metre (higher means smaller bumps). Because  $|\nu| \leq 1$  and  $\|\hat{n}\| = 1$ , no vertex moves farther than  $A$ , which makes object sizes testable. *Example:*  $A = 0.3$ ,  $\nu = 0.5 \Rightarrow$  the vertex moves 0.15 m outward. **Code:** Engine/Procedural/Deform.odin. **Other deformers:** taper scales  $(x, z)$  by a factor linear in height; twist rotates by  $\theta(y) = \Theta y/h$ ; bend wraps the axis on a circle of curvature  $1/R$ .

### G2. Two-bone inverse kinematics

$$\cos \alpha = \frac{a^2 + c^2 - b^2}{2ac}$$

Hip, knee and foot form a triangle with sides  $a$  (thigh),  $b$  (shin) and  $c = |\text{target} - \text{hip}|$ ; the law of cosines gives the hip angle  $\alpha$  between the target line and the thigh. The code clamps  $c$  and the cosine to  $[-1, 1]$  so an unreachable target gives a straight leg. *Example:*  $a = b = 0.45$ ,  $c = 0.8$ :  $\cos \alpha = 0.889$ ,  $\alpha = 27.3^\circ$ ; knee angle  $\cos \theta = (a^2 + b^2 - c^2)/2ab \Rightarrow 125.5^\circ$ . **Code:** Game/Characters/Ik.odin.

### G3. Fade curve, fractal noise and tiling

$$s(t) = 6t^5 - 15t^4 + 10t^3, \quad F(\mathbf{x}) = \frac{\sum_{i=0}^{n-1} 2^{-i} \nu(2^i \mathbf{x})}{\sum_{i=0}^{n-1} 2^{-i}}, \quad F(\mathbf{x} + P\mathbf{e}) = F(\mathbf{x})$$

**Fade.**  $s$  blends the four corner gradients; it has zero first and second derivative at 0 and 1 (so no grid lines show). The raw value is multiplied by  $\sqrt{2}$  so  $\nu \in [-1, 1]$  (unit gradients bound it by  $\sqrt{2}/2$ ). **Fractal sum.** Octave  $i$  has twice the frequency and half the strength; the denominator is the total weight, so  $F \in [-1, 1]$ . **Tiling.** Corner gradients are chosen from the cell index *modulo the period*  $P$  (doubling each octave), so the right edge equals the left edge: textures tile exactly (tested). *Example:*  $n = 4$ : weights  $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}$  sum to 1.875;  $s(0.5) = 0.5$ ,  $s(0.25) = 0.104$ . **Variants:** warping  $F(\mathbf{x} + s[F_1, F_2])$ , ridged  $\sum 2^{-i} (1 - |\nu_i|)^2 / \sum 2^{-i}$ , cellular (distance to the nearest random point). **Code:** Shaders/Include/Noise.glsl.

## G4. Normal map from a height map, and triplanar weights

$$h_x \approx \frac{3(h_{NE} + h_{SE} - h_{NW} - h_{SW}) + 10(h_E - h_W)}{32 \Delta N}, \quad \hat{n} = \frac{(-k h_x, -k h_y, 1)}{\|(-k h_x, -k h_y, 1)\|}$$

$$w_i = \frac{|n_i|^4}{\sum_{j \in \{x,y,z\}} |n_j|^4}$$

The surface direction tilts away from the slope. The *Scharr* filter weights the right column by 3, 10, 3 minus the left column; weights sum to 16 over two texels, hence  $32\Delta$ ; it treats diagonal and axis slopes equally.  $\Delta$ : one texel;  $N$ : cells per tile;  $k$ : bump strength. *Example*: ramp  $h = 0.7x$  gives  $h_x = 0.7$  exactly. Triplanar weights project the picture along three axes: for  $\mathbf{n} = (0.6, 0.8, 0)$  they are  $(0.24, 0.76, 0)$ . **Code**: `Shaders/Include/BakeMain.glsl`, `Triplanar.glsl`.

## Part C. Illumination models and lights

### L1. The lighting sum

$$L_o = \sum_{j=1}^{N_L} f(\mathbf{l}_j, \mathbf{v}) L_j \text{att}_j \text{sh}_j \max(0, \mathbf{n} \cdot \mathbf{l}_j) + \text{ambient} \cdot \text{AO} + \mathbf{e}$$

Light leaving a pixel is the sum over all  $N_L$  lights of: the illumination model  $f$  (share of light from  $\mathbf{l}$  that leaves toward the eye  $\mathbf{v}$ ), the light's colour times intensity  $L_j$ , its distance falloff  $\text{att}_j$ , its shadow  $\text{sh}_j \in [0, 1]$ , and Lambert's cosine  $\mathbf{n} \cdot \mathbf{l}$  (light spreads over a larger area at a slant); plus sky ambient light darkened by ambient occlusion AO, plus the surface's own emission  $\mathbf{e}$ . **Code**: `Shaders/Include/Lighting.glsl`, `Shaders/DeferredBase.glsl`. **Say**: "Every light adds model times colour times falloff times shadow times cosine; then add sky ambient and emission."

### L2. The six illumination models

$$f_{\text{Lambert}} = \frac{\mathbf{c}_d}{\pi}$$

$$f_{\text{Phong}} = \frac{\mathbf{c}_d(1 - F_0)}{\pi} + F_0 \frac{m + 2}{2\pi} (\mathbf{r} \cdot \mathbf{v})^m, \quad \mathbf{r} = 2(\mathbf{n} \cdot \mathbf{l})\mathbf{n} - \mathbf{l}$$

$$f_{\text{Blinn-Phong}} = \frac{\mathbf{c}_d(1 - F_0)}{\pi} + F_0 \frac{m + 8}{8\pi} (\mathbf{n} \cdot \mathbf{h})^m, \quad \mathbf{h} = \frac{\mathbf{l} + \mathbf{v}}{\|\mathbf{l} + \mathbf{v}\|}$$

$$f_{\text{Oren-Nayar}} = \frac{\mathbf{c}_d}{\pi} (A + B \max(0, \cos \Delta\varphi) \sin \alpha \tan \beta)$$

$$f_{\text{Cook-Torrance}} = \frac{(1 - F_l)(1 - F_v) \mathbf{c}_d}{\pi} + D V F$$

$$\langle \mathbf{n} \cdot \mathbf{l} \rangle_{\text{Subsurface}} = \text{clamp}\left(\frac{\mathbf{n} \cdot \mathbf{l} + w}{1 + w}, 0, 1\right), \quad w = 0.5$$

**Common symbols.**  $\mathbf{c}_d$  diffuse colour (metals have none);  $F_0$  base reflectance (0.04 for non-metals, the albedo for metals);  $m = \max(2/(\rho^2 + 10^{-4}) - 2, 1)$  shininess from roughness  $\rho$  (roughness  $0.5 \Rightarrow m = 6$ ). **Lambert**: matte;  $\mathbf{c}_d/\pi$  makes the total reflected light equal the colour (albedo  $0.5 \Rightarrow 0.159$ ). **Phong**: adds a highlight along the mirror direction  $\mathbf{r}$ ;  $(m + 2)/2\pi$  normalises it so a sharper highlight does not gain energy. **Blinn-Phong**: uses the half vector  $\mathbf{h}$  (cheaper, stable at glancing angles); normaliser  $(m + 8)/8\pi$ . **Oren-Nayar**: rough surfaces are many tiny Lambertian facets that scatter light back toward the source,

so they look flat instead of darkening at the edge.  $\sigma = \rho$ ,  $A = 1 - \frac{0.5\sigma^2}{\sigma^2+0.33}$ ,  $B = \frac{0.45\sigma^2}{\sigma^2+0.09}$ ,  $\alpha = \max(\theta_i, \theta_r)$ ,  $\beta = \min(\theta_i, \theta_r)$ ,  $\Delta\varphi$  the azimuth between light and view. For  $\sigma = 0.6$ :  $A = 0.739$ ,  $B = 0.360$  (Lambert is  $A = 1, B = 0$ ). **Subsurface**: the cosine ramps from 1 to 0 as  $\mathbf{n} \cdot \mathbf{l}$  goes  $1 \rightarrow -w$ , so light wraps past the shadow line like skin. *Example*:  $\mathbf{n} \cdot \mathbf{l} = 0.5 \rightarrow 0.667$ ;  $0 \rightarrow 0.333$ ;  $-0.25 \rightarrow 0.167$ ;  $-0.5 \rightarrow 0$ . **Cook–Torrance**: see L3. Where used: cloth and canvas Oren–Nayar, skin Subsurface, everything else Cook–Torrance. **Code**: `Shaders/Include/Brdf.glsl`.

#### How to say it in 20 seconds

“Lambert is matte; Phong and Blinn–Phong add a highlight; Oren–Nayar flattens rough cloth; Cook–Torrance is the physical default; Subsurface lets light wrap through skin.”

### L3. Cook–Torrance: distribution, Fresnel and visibility

$$D = \frac{\alpha^2}{\pi ((\mathbf{n} \cdot \mathbf{h})^2 (\alpha^2 - 1) + 1)^2}, \quad \alpha = \rho^2, \quad F = F_0 + (1 - F_0) (1 - \mathbf{v} \cdot \mathbf{h})^5$$

$$V = \frac{0.5}{(\mathbf{n} \cdot \mathbf{l}) \sqrt{(\mathbf{n} \cdot \mathbf{v})^2 (1 - \alpha^2) + \alpha^2} + (\mathbf{n} \cdot \mathbf{v}) \sqrt{(\mathbf{n} \cdot \mathbf{l})^2 (1 - \alpha^2) + \alpha^2}}$$

The surface is imagined as tiny mirrors.  $D$  (GGX) is the share of mirrors facing  $\mathbf{h}$  (only those reflect  $\mathbf{l}$  toward  $\mathbf{v}$ );  $F$  (Schlick) is how reflective a surface is at this angle (every surface becomes a mirror at glancing angles);  $V = G/(4\mathbf{n} \cdot \mathbf{l} \mathbf{n} \cdot \mathbf{v})$  is the Smith visibility (mirrors hiding each other). The diffuse term is scaled by  $(1 - F_l)(1 - F_v)$  because light that mirror-reflected cannot also scatter inside, keeping energy  $\leq 1$ . *Example*: roughness 0.5:  $\alpha = 0.25$ , peak  $D = 1/(\pi\alpha^2) = 5.09$ ;  $F_0 = 0.04$ :  $F = 0.07$  at  $60^\circ$ ,  $0.65$  at  $85^\circ$ . **Code**: `Shaders/Include/Brdf.glsl`.

### L4. Light sources: falloff and spot cone

$$\text{att}(d) = \begin{cases} \frac{(1 - (d/R)^4)^2}{d^2 + 1}, & 0 \leq d \leq R \\ 0, & d > R \end{cases} \quad c(\theta) = \begin{cases} 1, & \theta \leq \theta_i \\ \left( \frac{\cos \theta - \cos \theta_o}{\cos \theta_i - \cos \theta_o} \right)^2, & \theta_i < \theta < \theta_o \\ 0, & \theta \geq \theta_o \end{cases}$$

**Directional** (sun, moon): parallel rays, no falloff. **Point** (lamps): inverse-square law  $1/(d^2 + 1)$  (the  $+1$  keeps it finite at the lamp) times a smooth window  $(1 - (d/R)^4)^2$  that reaches exactly zero, with zero slope, at the range  $R$ , so the lamp can be drawn as a bounded sphere. **Spot** (floodlights): the point light times the cone factor  $c$ , where  $\theta$  is the angle from the spot axis,  $\theta_i$  inner and  $\theta_o$  outer. **Area** (ceiling panel): a  $4 \times 4$  grid of point emitters, each intensity/16 and weighted by its emission cosine. *Example*  $R = 14$  m:  $\text{att}(1) = 0.50$ ,  $(5) = 0.037$ ,  $(10) = 0.0054$ ,  $(14) = 0$ . Spot with  $14^\circ/26^\circ$ :  $c = 1$  at  $10^\circ$ ,  $0.33$  at  $20^\circ$ ,  $0$  at  $26^\circ$ . **Code**: `Shaders/Include/Lighting.glsl`.

### L5. Cascaded shadow maps

$$z_i = \lambda n \left( \frac{r}{n} \right)^{i/N} + (1 - \lambda) \left( n + (r - n) \frac{i}{N} \right), \quad i = 1, \dots, N$$

The view range from near plane  $n$  to far  $r$  is cut into  $N = 3$  slices. Logarithmic cuts (first term) keep shadow texels the same size on screen; even cuts (second term) cover the range uniformly;  $\lambda = 0.75$  blends them. *Example*:  $n = 0.1$ ,  $r = 100$ : cuts at 9.1, 24.2 and 100 m. A pixel uses the slice containing its depth, its lookup is moved along its normal (no self-shadow acne), and eight rotated percentage-closer taps soften the

edge. **Code:** Engine/Render/Shadows.odin, Shaders/Include/Shadow.glsl.

## Part D. Atmosphere, fog, tone map, world

### G5. Fog

$$C = T C_{\text{surface}} + (1 - T) C_{\text{haze}}, \quad T = e^{-k d}, \quad k = 3.5 \times 10^{-4} \text{ m}^{-1}$$

Each metre of air removes the same fraction of light (Beer–Lambert), so the surviving fraction  $T$  is exponential in distance  $d$ ; the rest is replaced by haze colour. *Example:*  $d = 100 \text{ m}$ :  $T = 0.966$ ;  $1 \text{ km}$ :  $0.705$ ;  $3 \text{ km}$ :  $0.35$ . The haze colour is the smooth atmosphere only (never stars or the moon, which caused the night streak bug).

**Code:** Shaders/DeferredBase.glsl.

### G6. ACES tone mapping

$$C_{\text{out}} = \text{clamp}_{[0,1]} \left( \frac{x (2.51 x + 0.03)}{x (2.43 x + 0.59) + 0.14} \right), \quad x = \text{exposure} \times \text{radiance}$$

Scene light can range from 0.01 to 40, a display shows 0 to 1. This fitted film curve compresses highlights smoothly, lifts dark detail and gives an S-shaped contrast; exposure rises from 1 (day) to 6 (night) like the eye adapting. *Example:*  $x = 0.18 \rightarrow 0.267$ ,  $x = 1 \rightarrow 0.804$ ,  $x = 4 \rightarrow 0.973$ . **Code:** Shaders/Include/Tonemap.glsl.

### G7. Sun height and cloud plane

$$\sin(\text{elevation}) = \sin \varphi \sin \delta + \cos \varphi \cos \delta \cos h, \quad h = 15^\circ (\text{hours} - 12), \quad \mathbf{P}_{xz} = \frac{H}{d_y} (d_x, d_z) + \mathbf{w} t$$

Spherical astronomy: latitude  $\varphi = 35^\circ$ , declination  $\delta = 15^\circ$ , hour angle  $h$  ( $15^\circ$  per hour). Noon:  $70^\circ$ ; 6:00 and 18:00:  $8.5^\circ$ ; midnight  $-40^\circ$  (the moon takes over). Clouds are a noise layer at height  $H$ : a ray with direction  $\mathbf{d}$  reaches it after  $H/d_y$  times the direction (similar triangles); adding  $\mathbf{w} t$  (wind velocity times time) slides the pattern. *Example:*  $\mathbf{d} = (0.8, 0.5, 0.33)$ ,  $H = 1500$ : point (2400, 995) m; with  $\mathbf{w} = (14, 5) \text{ m/s}$  the pattern moves (840, 300) m per minute. **Code:** Game/Gameplay/TimeOfDay.odin, Shaders/Include/Sky.glsl.

### G8. Tree sway

$$\Delta_x = 2s (0.04 + 0.1 g) \left( \frac{y}{9} \right)^2, \quad \Delta_z = 0.35 \Delta_x, \quad g = \frac{1}{2} + \frac{1}{2} \sin(1.3 t + 0.05 x + 0.03 z)$$

A tree is a beam fixed at the root, so it moves little at the bottom and most at the top: the shift grows with the *square* of height  $y$ .  $g \in [0, 1]$  is a gust travelling across the land;  $s$  is the sway strength (0.6 trunks, 2 foliage with flutter). *Example:* foliage  $s = 1$ , full gust,  $y = 9 \text{ m}$ :  $0.28 \text{ m}$ ; at  $4.5 \text{ m}$ :  $0.07 \text{ m}$ . **Code:** Shaders/Geometry.glsl.

### G9. Vehicle steering and engine power

$$\dot{\psi} = \frac{v \tan \delta}{L}, \quad \dot{v} = a_0 \left( 1 - 0.6 \frac{|v|}{v_{\text{max}}} \right)$$

Bicycle model: wheel base  $L$ , steering angle  $\delta$ , speed  $v$ ; the car turns on a circle of radius  $L/\tan \delta$ , so yaw rate =  $v/R$ . Thrust is power over speed, so acceleration is high from rest and fades with speed. *Example* (jeep,  $L = 3$ ,  $\delta = 0.55$ ,  $v_{\text{max}} = 18$ ,  $a_0 = 2.6$ ):  $\dot{\psi} = 2.04 \text{ rad/s}$  at  $10 \text{ m/s}$ ;  $v(t) = 30(1 - e^{-0.0867t})$  gives  $2.5 \text{ m/s}$  after one second and  $18 \text{ m/s}$  after  $10.6 \text{ s}$ . **Code:** Engine/World/GroundVehicle.odin.

## G10. Evaluation metrics

$$t = \frac{1000}{\text{fps}} \text{ ms}, \quad \varepsilon = \max_{\mathbf{u}} |f(\mathbf{u} + \mathbf{k}) - f(\mathbf{u})|, \quad \mathbf{k} \in \mathbb{Z}^2$$

Frame time for a 60 fps budget is 16.7 ms; our GPU work is 11.3 ms (the display waits for refresh). The tiling error  $\varepsilon$  is the largest difference between a texture and itself shifted by whole tiles; it must be exactly 0 for every material (TextureCheck).

## Likely questions

- **Why  $4 \times 4$  matrices for 3D?** The fourth coordinate lets translation be a matrix; chains collapse to one matrix.
- **Why is the transform order right to left?** Matrices multiply the column vector from the right, so the rightmost acts first.
- **Why the inverse transpose for normals?** It keeps normals perpendicular to the stretched surface (proof in T3).
- **Why a quintic fade and not linear?** Linear shows grid lines; the quintic has zero first and second derivative at the cell edges.
- **Why Blinn–Phong over Phong?** The half vector is cheaper and behaves better at glancing angles.
- **Why  $(1 - F_l)(1 - F_v)$  on the diffuse term?** Mirror-reflected light cannot also scatter inside; this keeps energy  $\leq 1$ .
- **Why cascades?** Near shadows need fine texels, far ones can be coarse;  $\lambda$  blends logarithmic and even cuts.
- **GPU 11.3 ms but frame 16.7 ms?** The display waits for the 60 Hz refresh; the rest is idle headroom.